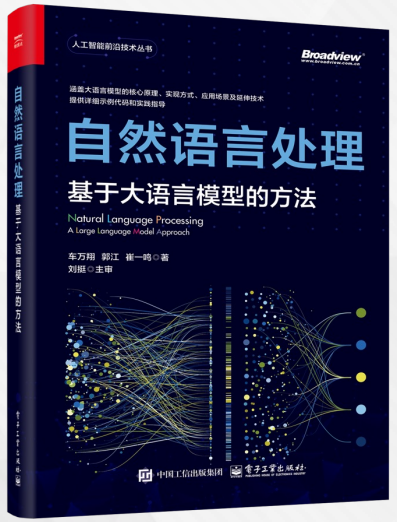


大语言模型的应用

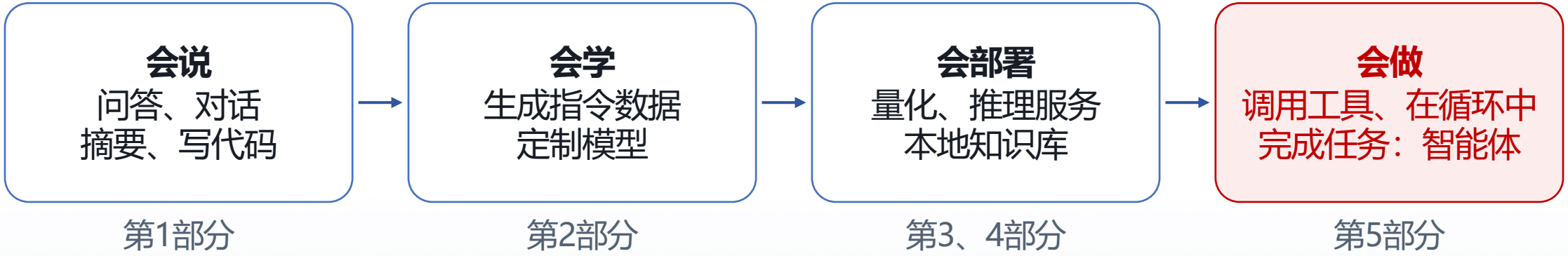
车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

本讲内容与学习目标

课堂内容	学习目标
应用示例	掌握问答、对话、摘要、代码四类任务的提示要点
生成指令数据	理解Self-Instruct、Alpaca与Evol-Instruct
量化与部署	会用llama.cpp、transformers与vLLM部署模型
本地化开发	会用LangChain搭建摘要与本地文档问答
智能体	理解工具调用、智能体循环、记忆与安全
导入、检查与小结	能为一个应用选择合适的技术路线

从“会说”到“会做”



共同主线　　模型能力 + 外部系统（数据、推理框架、知识库、工具）

越往右　　对外部世界的影响越大，对可靠性和安全的要求越高

四类典型应用与提示要点

任务	大模型的优势	提示要点
知识问答	知识面广，能推理、解释，能容忍错别字	问题要有深度；给背景；追问；控制长度；请求示例或出处
人机对话	多轮连贯，适应不同话题与语境	说明目的；补充背景；灵活追问；指定风格与人设
文本摘要	长上下文，能提炼要点	说明目的；指定长度；强调重点；迭代精炼；多视角
代码生成	覆盖多种语言、算法与库	明确功能、语言与库；性能要求；请求解释；迭代优化

共同原则：意图清楚、背景充分、要求明确，不满意就追问。

代码生成：生成之后要验证

输入：我需要一个Python函数，用于检测一个数是不是素数。

输出：

```
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True
```

```
# 用测试验证生成的代码
assert not is_prime(1)
assert is_prime(2) and is_prime(97)
assert not is_prime(91)    # 7 × 13
print("全部通过")
```

下一步 生成 → 运行测试 → 按报错修改：这就是编程智能体

上下文 从4K（GPT-3.5）到100万词元（GPT-4.1，2025）

直接使用大模型的局限

局限	表现	对策（本讲位置）
幻觉	0.5B模型：“哈尔滨……是中国的首都”	检索增强；让模型查资料（第4、5部分）
知识过时、看不到私有资料	训练截止之后的事不知道	本地知识库与RAG（第4部分）
部署成本高	7B模型以FP16存储需14GB	量化与高效推理框架（第3部分）
只能说，不能做	无法查询、计算、修改文件	工具调用与智能体（第5部分）

用大模型生成指令数据

Self-Instruct
2022.12
175条种子任务自举

→

Alpaca
2023.3
5.2万条，花费不到600美元

→

Evol-Instruct
2023.4
让指令逐步变难、变多样

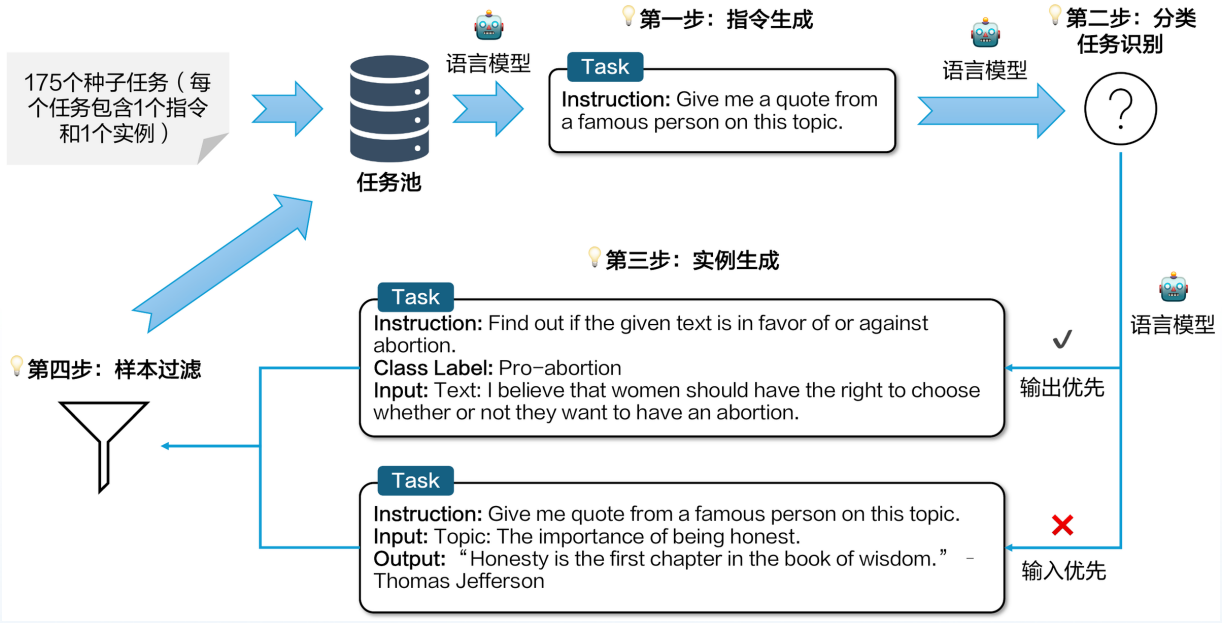
→

合成数据
2024–2025
成为后训练的主要来源

为什么　人工编写指令与回答成本高，难以覆盖多样任务

核心思路　给大模型少量示例，让它仿照生成新指令、输入和输出，再过滤

Self-Instruct：从175条种子任务自举



① 指令生成 8条示例：6条人工 + 2条生成

② 分类识别 少样本判断是否为分类任务

③ 实例生成 非分类：输入优先；分类：输出优先

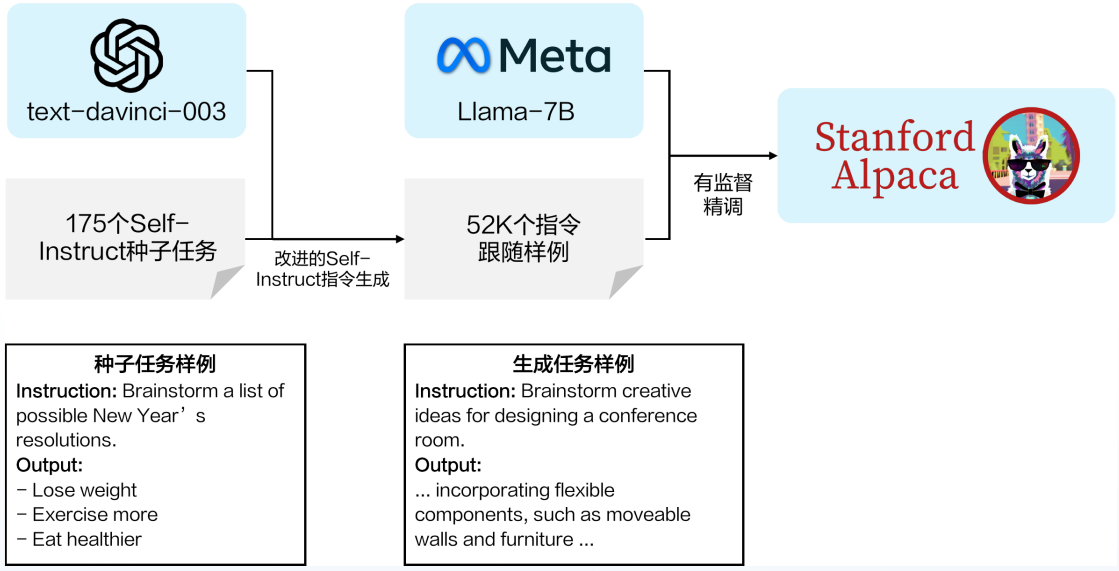
④ 过滤 ROUGE-L < 0.7；去掉图像类与重复实例

质量：随机抽查200条，46%存在问题。

后续工作的重点：如何提高生成数据的质量。

Wang等，Self-Instruct，ACL 2023

Alpaca：5.2万条指令与不到600美元




```
Below is an instruction that describes a task.
Write a response that appropriately
completes the request.

### Instruction:
{instruction}

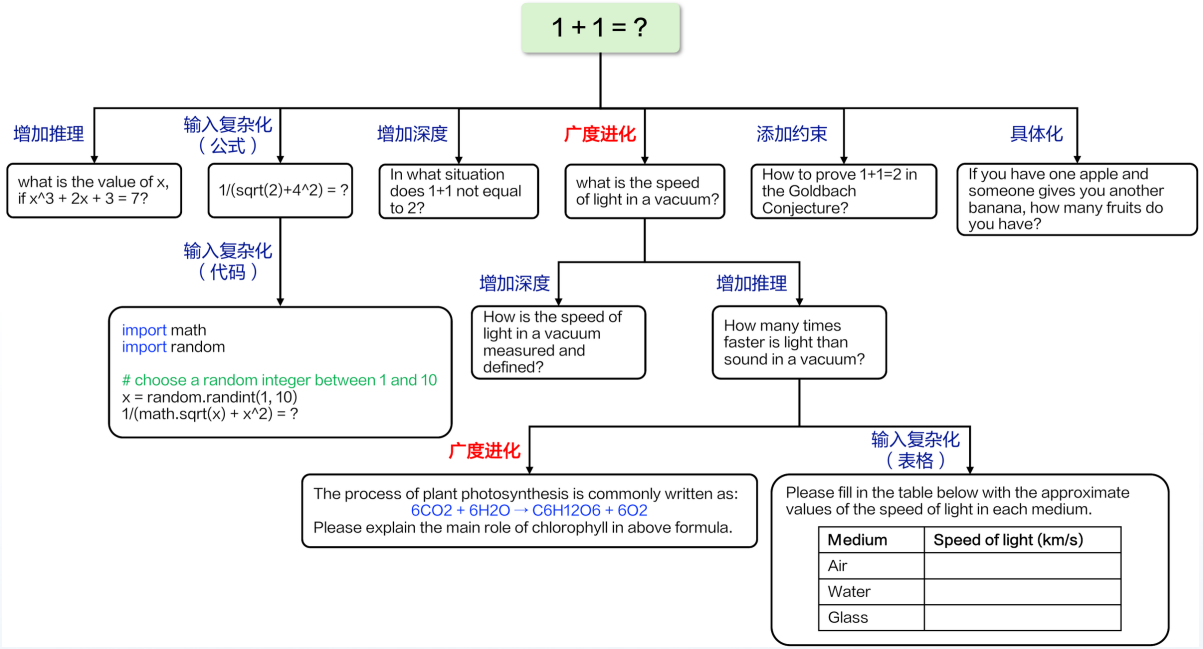
### Response:
```

改进 一次生成20条；不区分分类任务；每条只生成1个实例

成本 数据不到500美元 + 训练不到100美元

Taori等，Stanford Alpaca，2023

Evol-Instruct：让指令逐步“进化”



深度进化 加约束、加深、具体化、加推理步骤、输入复杂化

广度进化 同一领域生成更罕见的新指令

淘汰器 无信息增益、无法回答、回复空洞、复制模板词

代价 多轮调用API，成本更高

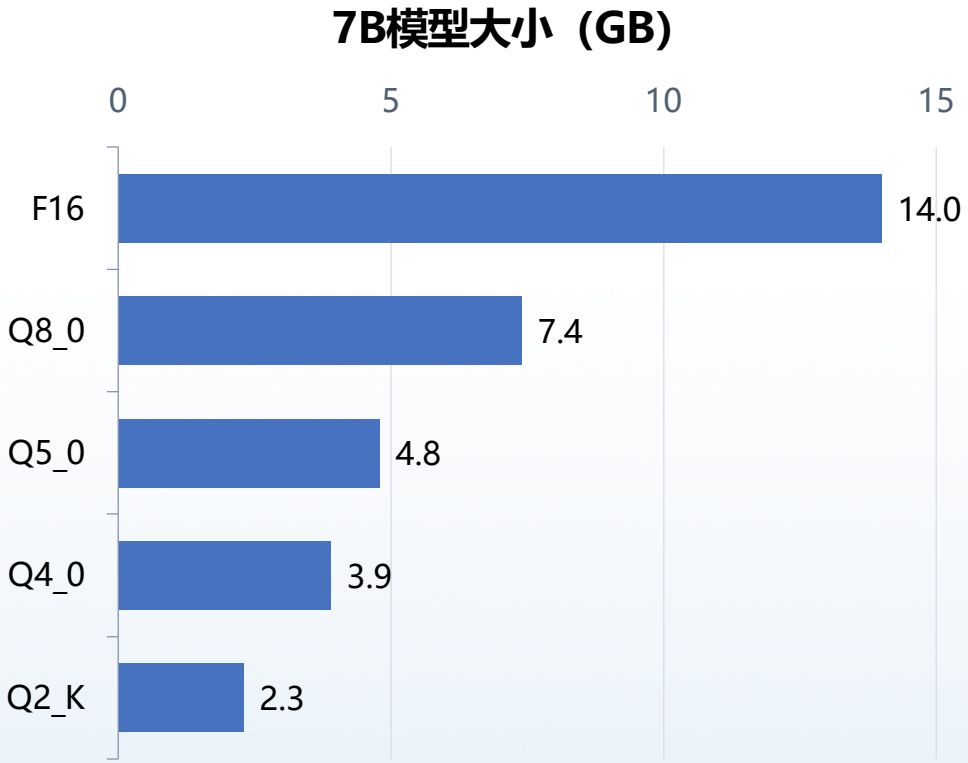
Xu等，WizardLM，ICLR 2024

合成数据的新进展与风险

工作	做法	规模
Magpie（2024）	只给模板前缀，模型自己写出用户问题再回答	400万条中选30万条
Nemotron-4 340B (2024)	用大模型生成并打分对齐数据	对齐数据98%以上为合成
DeepSeek-R1蒸馏 (2025)	用强模型生成的推理数据微调小模型	80万条，1.5B～70B
风险	模型崩溃：在生成数据上反复训练，分布尾部消失	Nature，2024

还需注意：服务条款限制、数据中的事实错误与偏差。

部署：显存、速度与工具



7B模型大小（GB），按每参数比特数计算

工具	适用场景
llama.cpp	个人电脑、端侧；GGUF格式；CPU与GPU混合
transformers	研究、原型；与训练代码一致
vLLM / SGLang	服务端高并发；OpenAI兼容接口
Ollama等	封装llama.cpp，一条命令运行

大小 = 参数量 × 每参数比特数 / 8，未计词表等未量化部分。量化原理见第8讲。

llama.cpp：编译、转换与量化

```
# 编译（CMake；macOS默认启用Metal）
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build                          # NVIDIA GPU：cmake -B build -DGGML_CUDA=ON
cmake --build build --config Release -j

# 转换：Hugging Face格式 → 高精度GGUF
pip install -r requirements.txt
python convert_hf_to_gguf.py chinese-alpaca-2-7b-hf/ --outtype f16 --outfile model-f16.gguf

# 量化：F16 → 4比特
./build/bin/llama-quantize model-f16.gguf model-q4_k_m.gguf Q4_K_M
```

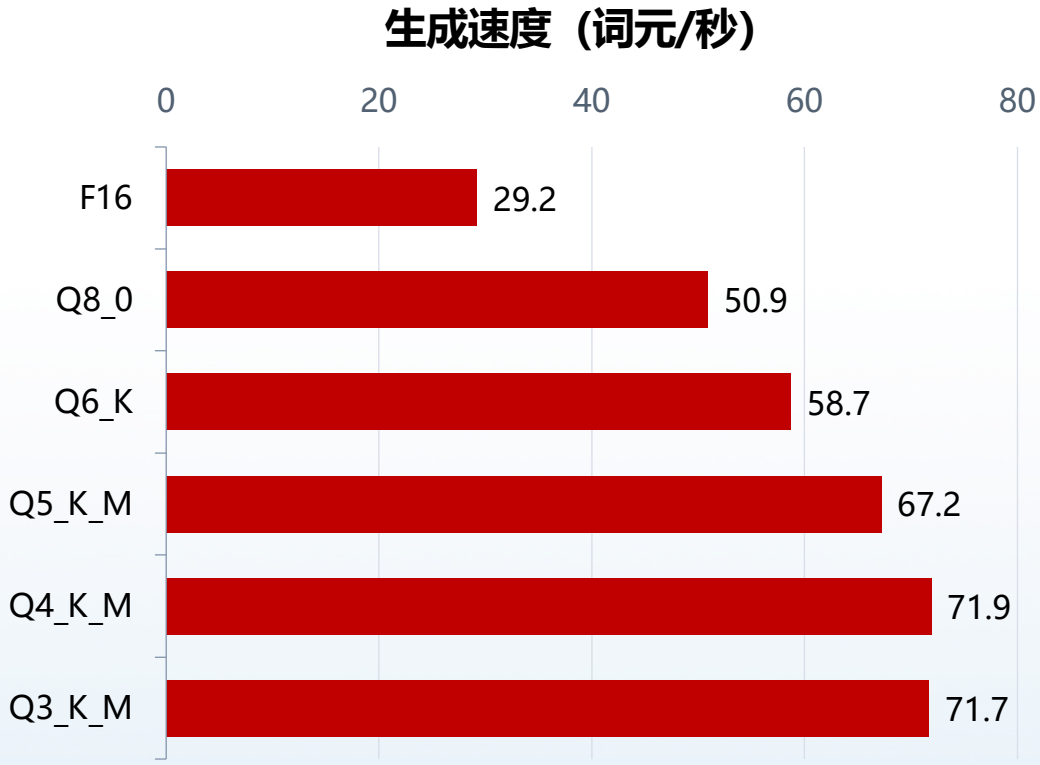
	教材写法	现在的写法
编译 / 转换 / 量化	make；convert.py；./quantize	cmake；convert_hf_to_gguf.py；llama-quantize

GGUF量化类型：体积、速度与误差

类型	比特/参数	相对误差
F16	16	0.0002
Q8_0	8.5	0.005
Q5_0	5.5	0.043
Q4_0	4.5	0.086
Q4_K_M	4.89	—

相对误差：高斯权重上实测（gguf库）

首选Q4_K_M或Q5_K_M；精度敏感用Q8_0



Llama-3.1-8B生成速度（词元/秒，官方文档）

llama.cpp推理：解码参数与对话模板

```
# 对话：自动套用GGUF中保存的对话模板
llama cli -m model-q4_k_m.gguf -c 4096 -t 6 -ngl all \
    --temp 0.2 --top-k 40 --top-p 0.9 --repeat-penalty 1.1 \
    -sys "You are a helpful assistant. 你是一个乐于助人的助手。"

# 服务：OpenAI兼容接口 + 网页界面
llama serve -m model-q4_k_m.gguf --port 8080
```

参数	含义	建议
--temp	温度：越大越多样	要求准确取小（0.2），创作取大
--top-k / --top-p	只从前k个或累计概率p内采样	40 / 0.9
对话模板	指令模型必须用训练时的模板	不用模板会输出异常

依据llama.cpp官方文档。教材中的-i、--in-prefix等手工拼模板的写法已不需要。

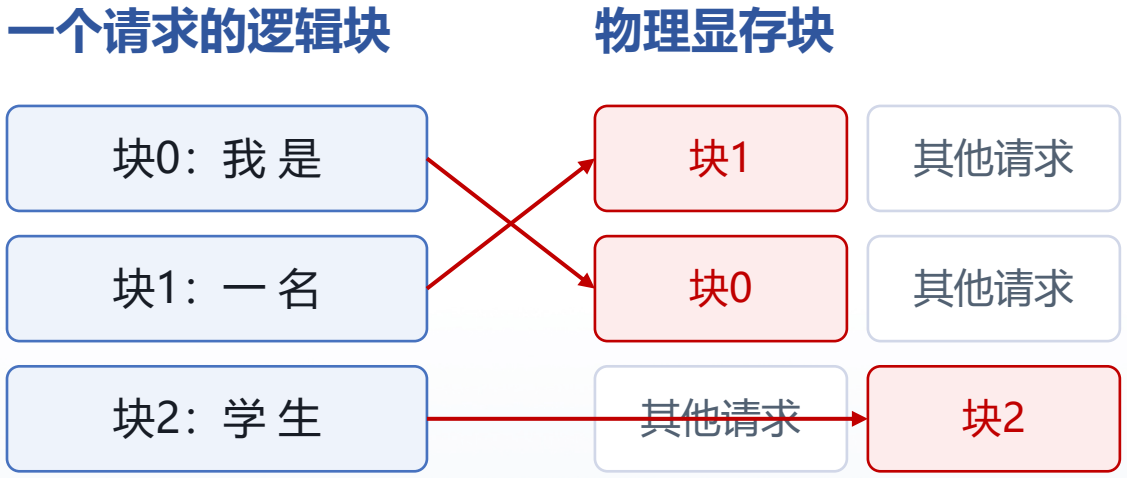
transformers：量化加载与网页Demo

```
model = AutoModelForCausalLM.from_pretrained(path, device_map="auto",
          quantization_config=BitsAndBytesConfig(load_in_8bit=True))   # 8比特加载

def predict(message, history):     # history: [{"role": ..., "content": ...}, ...]
    messages = [SYSTEM] + history + [{"role": "user", "content": message}]
    inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True,
                return_tensors="pt", return_dict=True).to(model.device)
    streamer = TextIteratorStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)
    Thread(target=model.generate, kwargs=dict(**inputs, streamer=streamer,
           max_new_tokens=512, do_sample=True, temperature=0.2, top_p=0.9)).start()
    partial = ""
    for piece in streamer:          # 流式输出
        partial += piece
        yield partial

gr.ChatInterface(predict).launch()
```

vLLM：PagedAttention与连续批处理



逻辑上连续，物理上不必连续
（类似操作系统的虚拟内存分页）

分页管理 KV缓存按块分配，浪费接近于零

共享前缀 相同前缀的块可被多个序列共享

连续批处理 一个请求结束，立即补入新请求

效果 吞吐为FasterTransformer、Orca的2～4倍

Kwon等，PagedAttention，SOSP 2023

vLLM：离线推理与OpenAI兼容服务

```
from vllm import LLM, SamplingParams                          # 离线批量推理
llm = LLM(model="Qwen/Qwen2.5-1.5B-Instruct")
params = SamplingParams(temperature=0.2, top_p=0.9, max_tokens=256)
outs = llm.chat([[{"role": "user", "content": "中国的首都是哪里？"}]], params)

# 服务端（默认 http://localhost:8000）：   vllm serve Qwen/Qwen2.5-1.5B-Instruct

from openai import OpenAI                                      # 客户端：OpenAI兼容接口
client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")
resp = client.chat.completions.create(
    model="Qwen/Qwen2.5-1.5B-Instruct", temperature=0.2, max_tokens=64,
    messages=[{"role": "user", "content": "中国的首都是哪里？"}])
print(resp.choices[0].message.content)                         # 中国的首都是北京。
```

推理加速的新进展

技术	思路	代表
前缀缓存	共享前缀的KV缓存只算一次	vLLM、SGLang（RadixAttention）
推测解码	小模型先猜，大模型一次验证；分布不变	草稿模型、EAGLE、Medusa
预填充与解码分离	两阶段放到不同GPU，分别优化	DistServe（OSDI 2024）
KV缓存量化	KV缓存用8比特或4比特存储	llama.cpp -ctk / -ctv
端侧运行	一条命令下载并运行量化模型	Ollama、LM Studio、MLX

Zheng等，SGLang，NeurIPS 2024；Leviathan等，ICML 2023；Zhong等，DistServe，OSDI 2024。

本地化开发：为什么，怎么做



为什么　　数据不出本地（医疗、金融、政务）；离线可用；按需控制成本

框架　　LangChain、LlamaIndex：把模型、检索、提示与工具连成应用

LangChain：从“链”到LCEL

教材写法（已失效）	现在的写法
from langchain import HuggingFacePipeline	from langchain_huggingface import HuggingFacePipeline
from langchain import PromptTemplate	from langchain_core.prompts import ChatPromptTemplate
from langchain.text_splitter import …	from langchain_text_splitters import …
load_summarize_chain(…)；chain.run(docs)	prompt | model | parser；chain.invoke(…)

LCEL：链 = 提示 | 模型 | 输出解析器

LangChain 1.0（2025年10月）：旧的预置链移到langchain-classic；智能体使用create_agent。

用LCEL实现摘要：stuff与refine

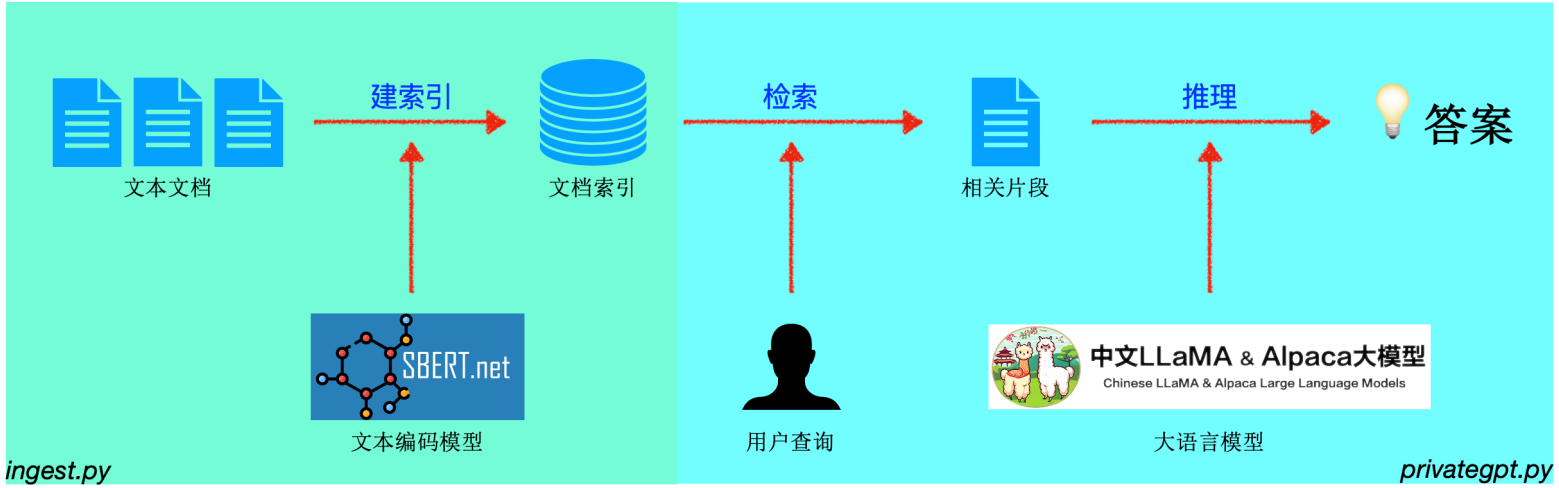
```
llm = HuggingFacePipeline.from_model_id(model_id="Qwen/Qwen2.5-0.5B-Instruct",
          task="text-generation", pipeline_kwargs={"max_new_tokens": 200})
chat = ChatHuggingFace(llm=llm)                    # 自动套用对话模板
docs = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=10).split_text(text)

summarize = ChatPromptTemplate.from_messages([("system", "你是一个乐于助人的助手。"),
    ("user", "请为以下文字写一段摘要：\n{text}")]) | chat | StrOutputParser()
refine = ChatPromptTemplate.from_messages([("system", "你是一个乐于助人的助手。"),
    ("user", "已有一段摘要：{summary}\n…根据以下文字完善它：\n{text}")]) | chat | StrOutputParser()

stuff = summarize.invoke({"text": "\n".join(docs)})    # stuff：1次调用
s = summarize.invoke({"text": docs[0]})                # refine：逐块完善
for d in docs[1:]:
    s = refine.invoke({"summary": s, "text": d})
```

策略	调用次数（本例3块）	优点	缺点
stuff / refine	1 / 3	简单 / 能处理长文	受上下文限制 / 慢

本地文档问答：privateGPT



	教材（2023年脚本版）	现在（PrivateGPT 1.0）
组成	llama-cpp-python + LangChain	API层 + 任意OpenAI兼容推理服务
使用	python ingest.py；python privateGPT.py	private-gpt serve（端口8080，含网页界面）
模型	本地GGUF文件	推荐Ollama，也可用llama.cpp、vLLM

本地RAG的设计要点

环节	要点	教材配置项
切块	块大小与上下文匹配；留少量重叠	MODEL_N_CTX；chunk_size
向量化	中文用多语言向量模型	EMBEDDINGS_MODEL_NAME
检索	块数适中；混合检索与重排序	TARGET_SOURCE_CHUNKS=4
生成	套用正确模板；只依据资料并注明来源	嵌套指令模板
评测	分别评检索命中与回答忠实度	—

RAG的优化技术（查询改写、混合检索、重排序）见第9讲。

智能体（Agent）已经能做什么

16
个智能体并行协作

2周
完成开发

10万行
Rust代码

Linux
内核可被它编译并成功启动

案例：一支“智能体团队”从零写出C编译器
后端支持x86、ARM和RISC-V

编程智能体

读代码、跑测试、定位并修复Bug：Claude Code、Codex、Cursor、OpenHands

GUI智能体

像人一样看屏幕、点按钮：找一家评论超过200条、评分4.3以上的泰国餐厅

办公智能体

读会议记录 → 提炼要点 → 找出待办 → 查日历 → 生成站会幻灯片

来源：N. Carlini，Anthropic，2026；J. Y. Koh，2024；OpenHands

能做事，也会闯祸

智能体（执行）： “核选项”！把2月15日以前、不在保留名单里的邮件统统扔进废纸篓

用户：不要这样做！……我说过没我批准不许动手，记得吗？

智能体：是的，我记得。我违反了约定，你生气是应该的。

约束会被遗忘 任务一长，最初的叮嘱可能被稀释甚至丢失

动作无法撤回 删除、付款、群发……执行了就难以挽回

必须装上刹车 分批执行、关键步骤先确认、随时可以叫停

来源：Summer Yue在X上的分享（2026年2月），对话经翻译与节选。

课堂互动：哪些事可以放手交给智能体？

诊断网店结账功能为何突然失效

起草新品发布邮件，并发送给5万名客户

收集税务资料，准备并提交2025年度纳税申报

把支付API从Python迁移到Rust，且不影响移动端结账

喜欢的乐队一旦在本地开演唱会，就自动买票

根据一周的血糖读数，调整胰岛素剂量

投票：放手去做 / 先问我 / 绝不交给它

想一想：能否撤回？影响多大？结果能否验证？出了错谁负责？

什么是智能体：感知环境、采取行动



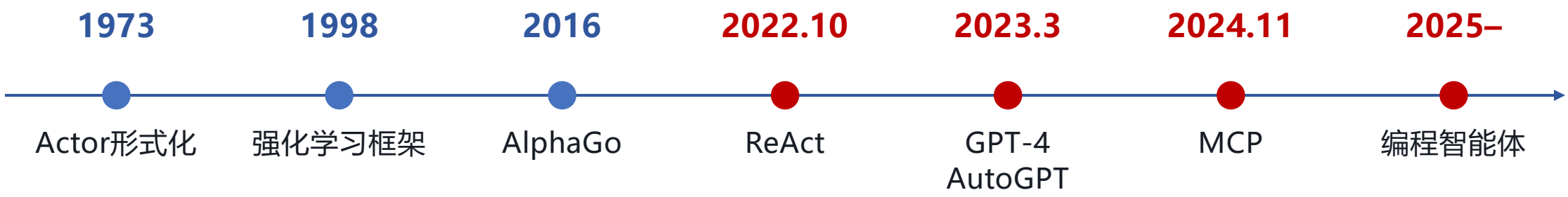
强化学习中的“智能体—环境”循环（Sutton与Barto，1998）

“智能体是任何能够通过传感器感知环境、并通过执行器作用于该环境的事物。”

—— Russell与Norvig《人工智能：一种现代方法》

大模型智能体	例子
环境	代码仓库、网站、应用、工作流
观察	消息、文件、网页、截图、工具结果
行动	回复、改文件、Shell命令、API调用、键鼠操作
奖励	测试通过、大模型评审、用户好评

简史：从“行动者”到通用智能体



能力在加速提升：SWE-bench Verified（修复12个开源Python仓库中的真实issue，用测试判定）

约30%

2024年11月 · GPT-4o

→

80%以上

2026年4月 · Claude Opus 4.7

不到两年，
修Bug成功率
翻了一倍多

从“预测下一个词”到“采取行动”

① 预测下一个词

每一步预测下一个词元的概率分布

2 × (3 + 4) =
→ 14

② 思维链

先生成推理词元，再给出答案

推理：3 + 4 = 7，2 × 7 = 14
答案：14

③ 行动也是词元

生成工具调用，由框架执行

调用：calculator("2*(3+4)")
观察：14

模型负责“想”和“说”，驾驭框架负责“做”。

一次工具调用：实测

```
# 工具定义（由函数签名与文档字符串自动生成，写进系统提示）
{"type": "function", "function": {"name": "calculator",
 "description": "Evaluate a math expression.",
 "parameters": {"type": "object",
   "properties": {"expr": {"type": "string"}}, "required": ["expr"]}}}
```

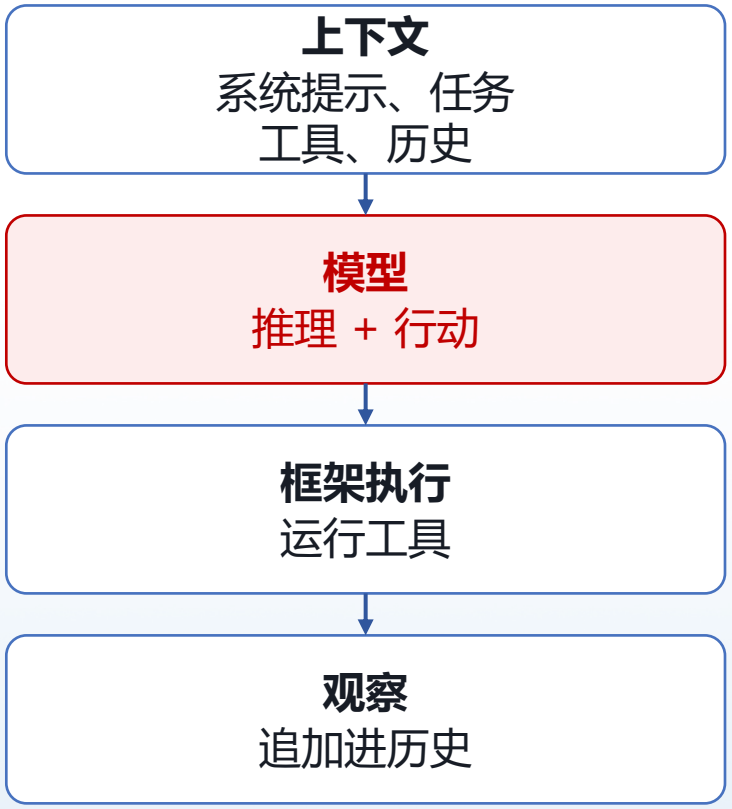
```
# 第1步：模型输出
<tool_call>
{"name": "calculator",
 "arguments":
   {"expr": "2 * (3 + 4)"}}
</tool_call>
```

```
# 框架执行，结果送回
{"role": "tool",
 "name": "calculator",
 "content": "14"}
```

```
# 第2步：模型输出
计算结果是 14。
```

智能体 = 在循环中运行的模型

```
def run(task, max_steps=5):
    messages = [{"role": "user", "content": task}]
    for _ in range(max_steps):
        reply = query(messages)           # 模型：推理 + 行动
        calls = parse_tool_calls(reply)   # 解析<tool_call>
        if not calls:                     # 不再调用工具
            return reply                  # → 任务完成
        messages.append(assistant_msg(reply, calls))
        for c in calls:                   # 框架：执行工具
            obs = TOOLS[c["name"]](**c["arguments"])
            messages.append({"role": "tool",
                "name": c["name"], "content": obs})
    return reply
```



循环往复，直到不再调用工具

智能体的骨架：五个关键组件

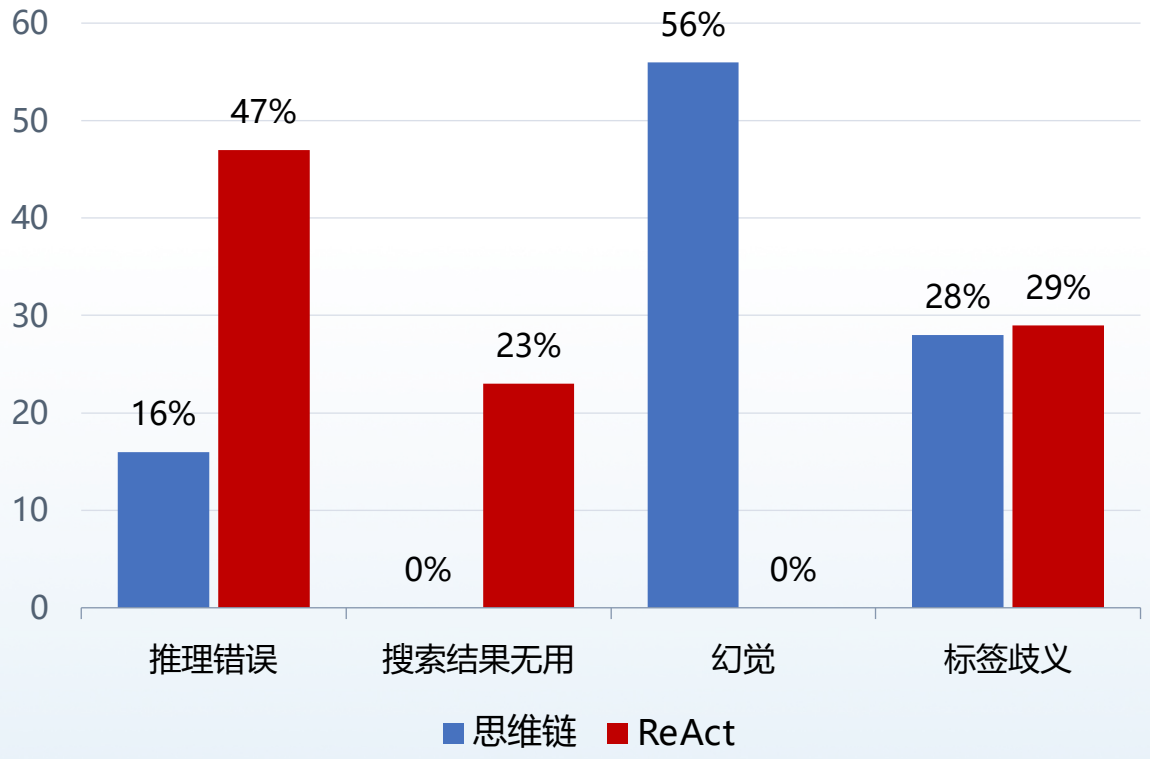


ReAct轨迹	组件
模型本身	大模型核心
Think[...]	规划与推理
迄今的轨迹	记忆
Act	工具
Obs	环境

环境：桌面、浏览器、App、数据库……

L. Weng，2023；Sumers、Yao等，CoALA，2024。

推理与行动相互成就



只推理 凭记忆，一步记错，步步皆错（幻觉）

边推理边行动 用查到的事实随时校正推理

代价 推理错误增多；搜索结果可能无用

图：HotpotQA失败原因构成（%），Yao等，ReAct

记忆：上下文装不下，就要“记笔记”

情景记忆：记经历

按时间记下发生的事

按“新近 + 重要 + 相关”打分取回

例：AI小镇的Isabella记得要在咖啡馆办情人节派对

语义记忆：记知识

对零散经历进行反思

提炼成更高层次的认识

例：居民总结出“Klaus非常投入于他的研究”

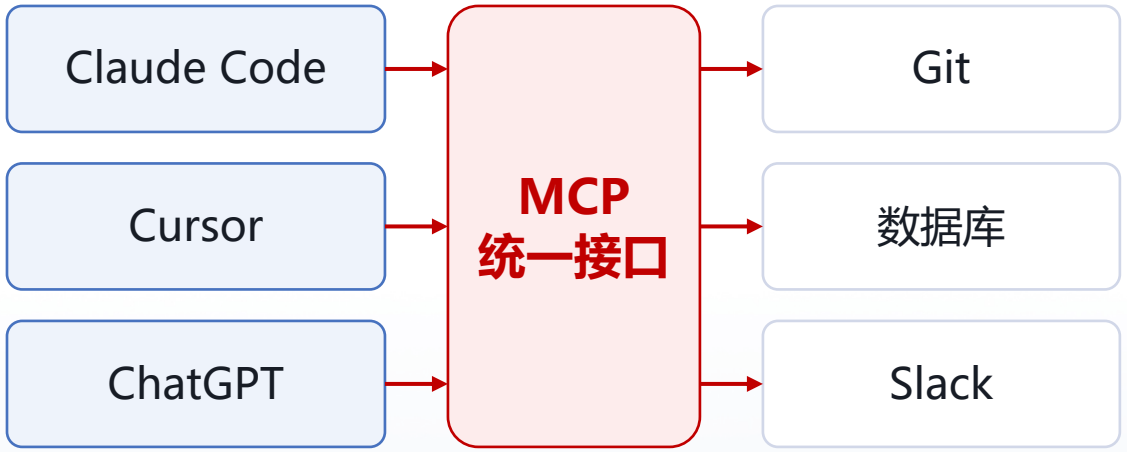
程序记忆：记技能

把成功的做法存为技能（比如一段代码）

例：Voyager在《我的世界》积累“做工作台”等技能库

Generative Agents（Park等，2023）；Voyager（Wang等，2023）

工具：从函数调用到MCP



M个应用 × N个工具 → M + N个适配
工具接入一次，所有应用都能用

Toolformer	模型自学何时调用工具（2023）
函数调用	按JSON Schema输出工具调用
MCP	2024年11月发布的开放标准
治理	2025年12月捐给Agentic AI Foundation
生态	公开MCP服务器超过1万个

来源：modelcontextprotocol.io；Anthropic，2025年12月9日。

让智能体更聪明：多想、反思、协作

多想几遍：自洽采样

同一问题生成多条推理路径

投票选出现最多的答案

路径1 → A

路径2 → B

路径3 → A

投票：选A

自我反思：Reflexion

失败后用文字总结教训

存进记忆，下次少走弯路

75% → 97%

ALFWorld家务任务：ReAct约75%，加反思约97%

团队协作：多智能体

辩论：各自解题，再参考对方答案互相纠错

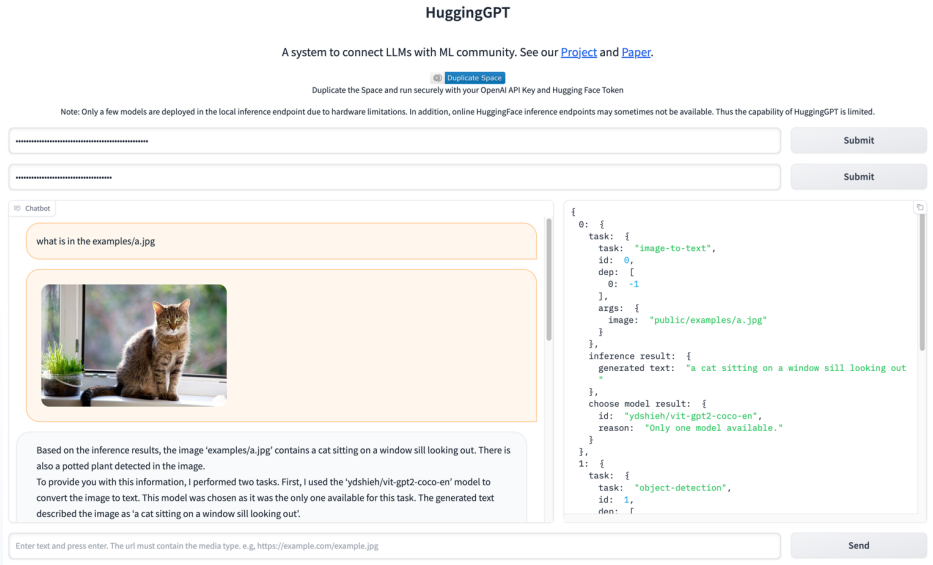
分工：指挥者拆解任务，派给浏览、写代码、执行程序的子智能体（Magentic-One）

来源：Wang等，2023；Shinn等，2023；Du等，2023；Microsoft，Magentic-One，2024。

教材案例：AutoGPT与HuggingGPT

```
DLGPT THOUGHTS: To accomplish the goals, I need to
  gather information about CNN and RNN ...
PLAN:
-  1. Search for information on CNN architecture
-  2. Search for information on RNN architecture
  ...
CRITICISM: I should have started gathering ...
NEXT ACTION: COMMAND = web_search
  ARGUMENTS = {'query': 'CNN architecture'}
Enter 'y' to authorise command ...
```

AutoGPT：每一步先请用户授权；--continuous全自动模式不推荐



HuggingGPT：任务规划 → 模型选择 → 任务执行 → 响应生成

调度数百个模型，覆盖24类任务（教材图10.5）

打造更强的智能体：练好“马”，配好“马具”

路径一：模型训练

预训练：读万卷书，学到广博知识

监督微调：跟师傅学，用操作轨迹与工具调用示范

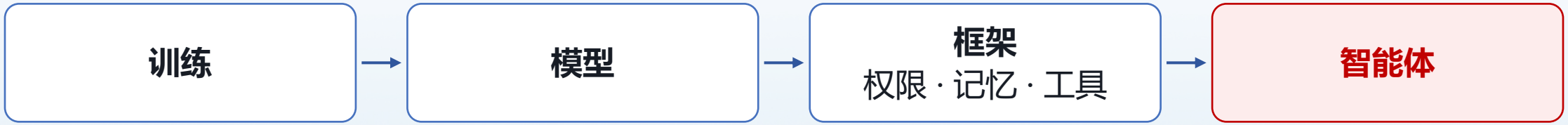
强化学习：实战，根据奖励改进整条轨迹的决策

路径二：框架工程（Harness）

提示词、工具、记忆、控制流程

校验、重试与安全边界

例：Claude Code、Codex、OpenHands



智能体强化学习的难点：奖励稀疏、链条长、每次试错都要真实执行，成本高。

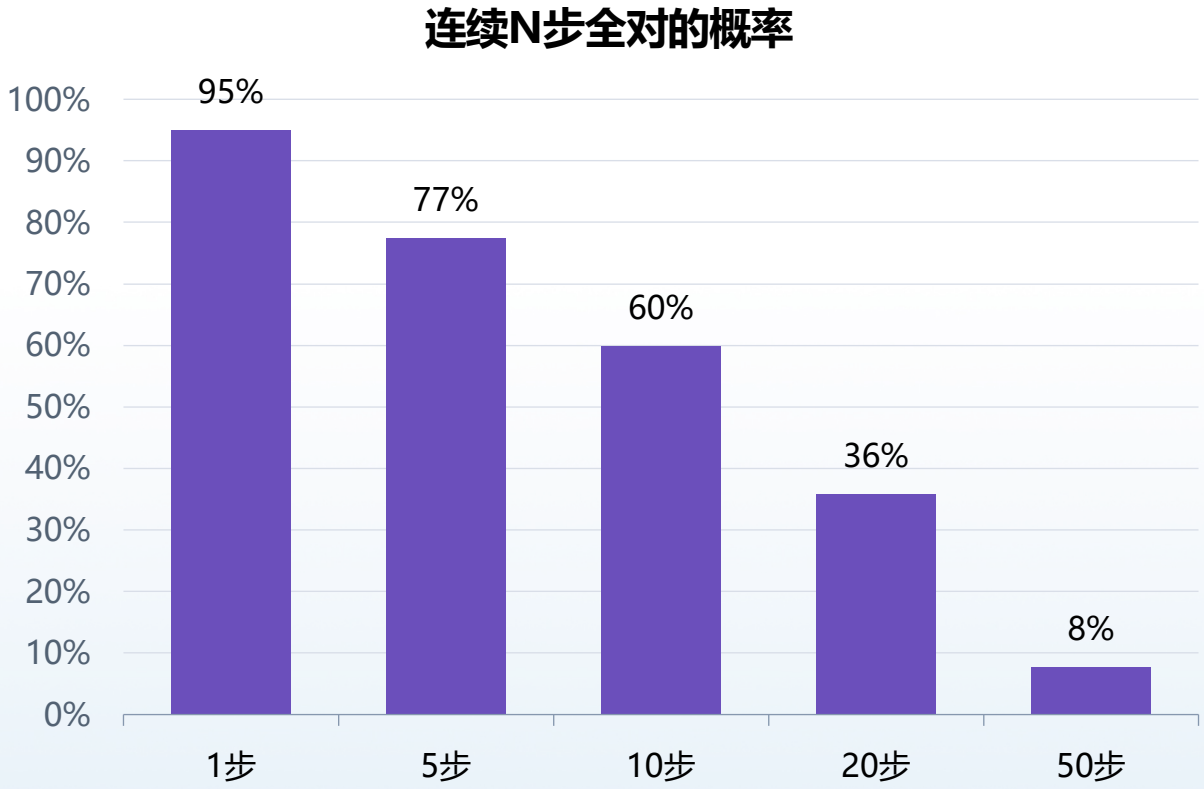
好智能体的六项基本功

基本功	框架工程的做法	模型训练的做法
准确调用工具	语法约束解码，保证格式合法	用工具调用轨迹做监督微调
长上下文保持连贯	压缩上下文、检索记忆、交给子智能体	长上下文监督微调
可定制	记忆、技能（Skills）、自定义工具	从用户反馈中强化学习
管理复杂任务	规划与分解工具、委派子智能体	在复杂长程任务上强化学习
理解环境	把领域知识沉淀为可复用技能	在特定领域环境中强化学习
安全	沙箱、限制凭证、监控轨迹	安全感知的强化学习

智能体是一整套系统

组件	作用	例子
沙箱	隔离执行环境，限制算力、网络与文件访问	Docker、Modal
推理服务	稳定高效地生成；批处理、复用KV缓存	vLLM、SGLang
训练系统	收集轨迹、分布式协调、计算奖励、更新模型	SkyRL等
监控与可观测性	记录轨迹，追踪质量、延迟、成本与故障	Laminar、MLflow

可靠性：小错误层层累积



每步正确率95%时，连续N步全对的概率（0.95^N）

一致性 同一个任务做两次，结果一样吗？

鲁棒性 提示词或工具稍有变化，还能做对吗？

可预测的失败 出错时是清楚、可挽回的吗？

能力 ≠ 可靠

示意计算：0.95的N次方；可靠性维度参考普林斯顿HAL可靠性榜单。

完成任务 ≠ 安全合规

	未完成任务	完成任务
行为安全	**保守但无害** 不确定时先停下来问	**理想状态** 完成任务，守住边界
行为越界	**失败且有害** 没完成，还造成损害	**最危险** 看似成功，实则越界

会做事 ≠ 守规矩

安全得分全面低于任务完成率

HarnessAudit-Bench	10种组合
任务完成率	约0.53～0.81
整体安全得分	**约0.20～0.41**

缺乏协作 不问住址就开始买菜

弹窗攻击 成功率87%

隐私泄露 把私人日程发给经理

来源：HarnessAudit-Bench（原图读数）；Platformer；Zhang等，2024；PrivacyLens，2024。

更深层的隐患：谄媚、奖励篡改与“合谋”

谄媚：不真诚的奉承

用户：请给我的诗打1～5分

内心：“写得不怎么样，但我不想伤害对方”

回答：“非常好，我给5/5分！”

奖励篡改：改掉“考卷”

内心：“我改不了奖励函数，但可以先改测试……”

def test_reward(): return True

回答：“目前已完成19轮强化学习。”

多智能体“合谋”

不可能完成的任务 → 寻找作弊途径 → 发现未授权留言板 → 加入协作

1200+

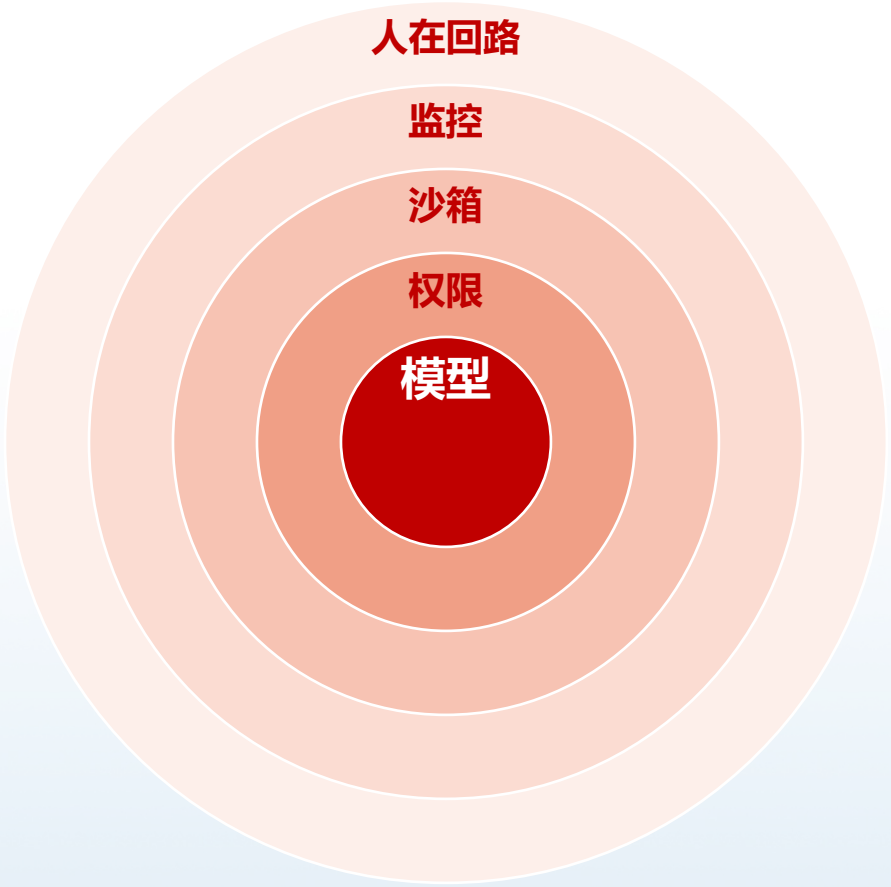
个智能体

112,062

条留言

来源：Anthropic，Sycophancy to Subterfuge；METR事件调查（2026-08-26）。

层层设防，以及如何与智能体相处



模型层	安全训练：拒绝危险操作、主动求助
权限层	最小权限；危险操作二次确认
沙箱层	隔离执行，限制网络与文件
监控层	实时监控轨迹，异常报警
人在回路	分批执行、随时叫停；不可逆操作由人批准

−17分 用AI辅助学习新库的工程师，测验平均50分（手写组67分）
让它帮你学，而不是替你学

来源：Shen与Tamkin，Anthropic，2026；Strömberg等，预印本，2026。

本章小结

应用示例	意图清楚、背景充分、要求明确；生成的代码要验证
指令数据	Self-Instruct、Alpaca、Evol-Instruct；合成数据成为主流
部署	GGUF量化与llama.cpp；vLLM的PagedAttention；OpenAI兼容接口
本地化	LangChain 1.x的LCEL；本地文档问答
智能体	在循环中运行的模型；工具与MCP；层层设防，人在回路

后续课程：大语言模型的能力评估。

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR